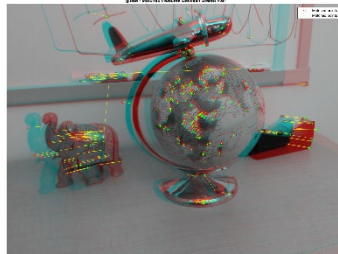
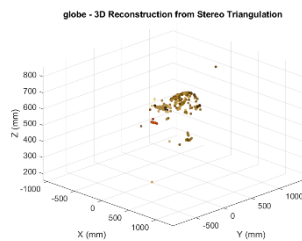
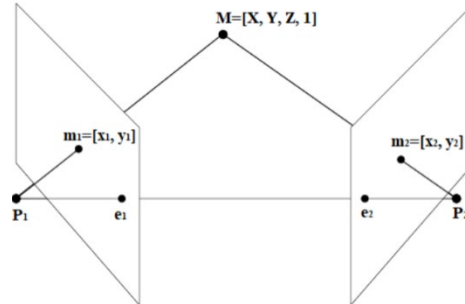


3D Point Cloud Generation

1. Introduction:



In this assignment we are implementing linear back-projection or triangulation according to the textbook (Hartley-Zisserman, Multiple View Geometry 2nd Edition Chapter 12). The purpose of this is to produce a 3D projection from 2 Images.



Given a pair of images that are a stereo view of a scene, we use matching points to triangulate 3D points. All these 3D points can be used to create a point cloud model of the scene. The greater the number of matching points, the greater the density of the point cloud.

2. Description of the methods implemented for accomplishing 3D Point Cloud Generation

In order to create this Point Cloud Projection, these steps were taken:

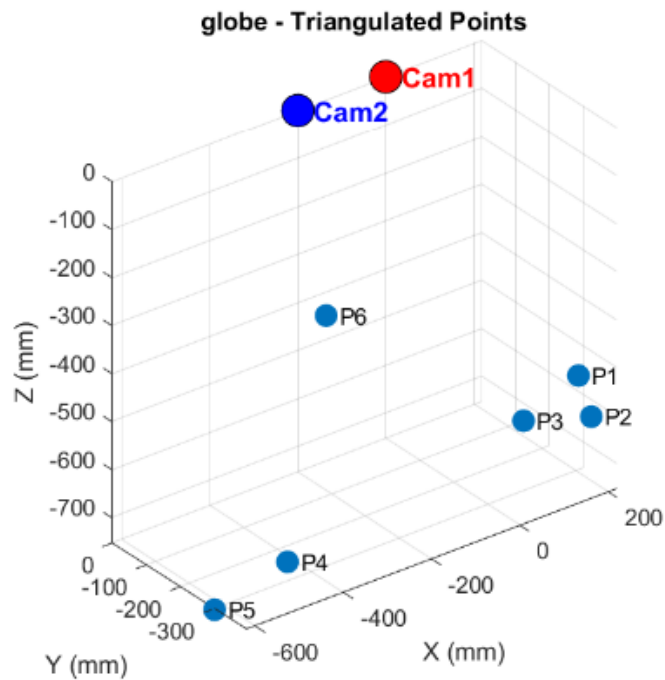
1. Load Stereo Images and Calibration Data
2. Manually select Points in the first image

3. Manually select the Matching points in the second image. The points in the first and second images should be the same part of the scene.
4. Build projection matrices
5. Compute Normalizing Transform
6. Transform Points and Cameras
7. Triangulate Points

3. Results Showing Input Images and Resultant 3D Scatterplot:

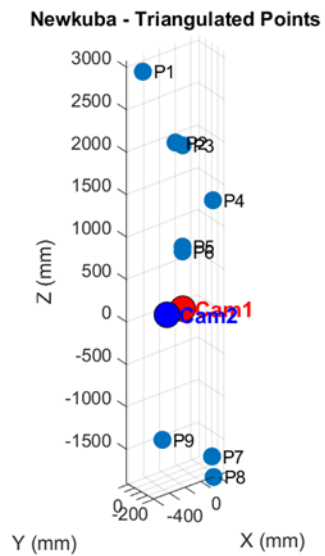
Globe Images

Signal to Noise Ratio between the Link Back Projection Function and Matlab's Native Triangulate Function: **0.60 dB**



Newkuba Images

Signal to Noise Ratio between the Link Back Projection Function and Matlab's Native Triangulate Function: **12.33 dB**

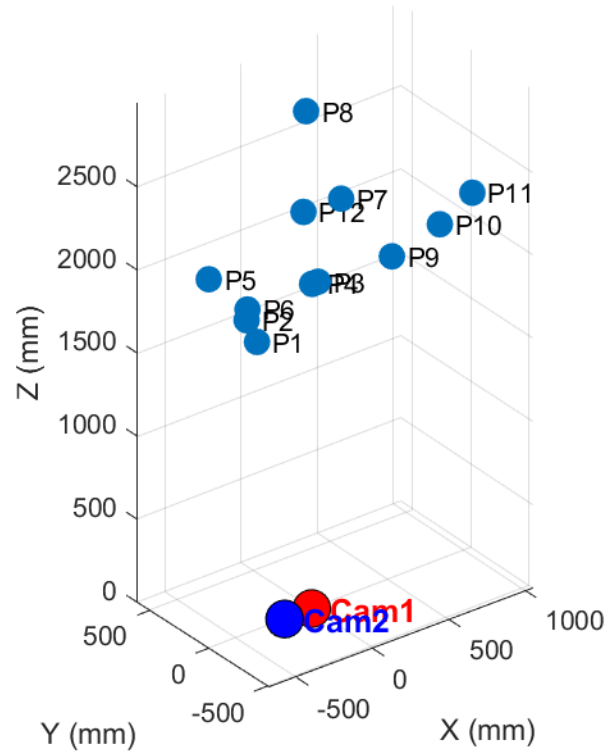


Piano Images

Signal to Noise Ratio between the Link Back Projection Function and Matlab's Native Triangulate Function: **0.55 dB**

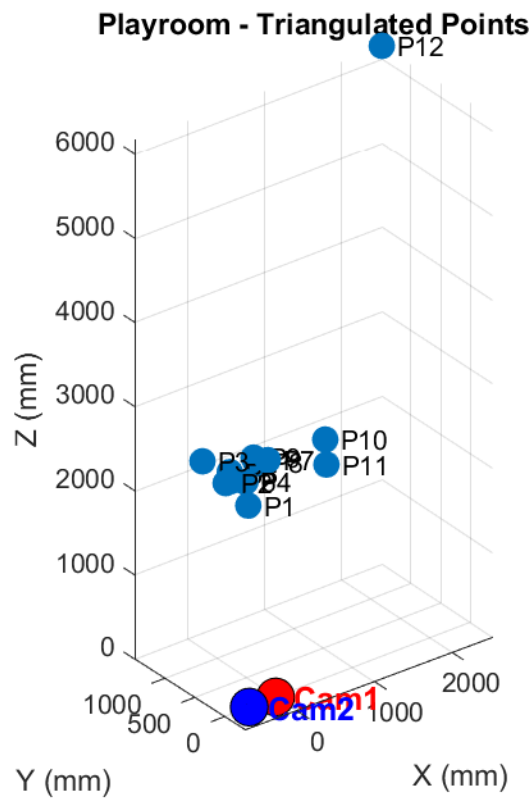


Piano - Triangulated Points



Playroom

Signal to Noise Ratio between the Link Back Projection Function and Matlab's Native Triangulate Function: **5.03 dB**



Interpretation of Results

The SNR values between the implemented Link Back Projection Implementation and the inbuilt Triangulation Function is low. This is most likely because of the difference in the number of points selected. Due to the fact that I had to manually select matching points with my implemented algorithm, I was not able to select that many points. However, Matlab's Triangulate Algorithm automated the process and so many more points were selected.

It seems that a big part of having this algorithm produce a good point cloud is to be able to choose many points and to choose them accurately in an autonomous manner. With many point selected, you would be able to get a more visually recognizable image.